

Ship-Radar 对外 UDP 接口协议（以现行代码为准）

适用工程：DispCtrl 的 ship-radar 分支

协议依据：Basic/MarineProtocol.h、Controller/MarineRadarManager.cpp、Controller/controller.cpp、mainPanel/mainoverlaylayout.cpp。

Word 参考：船用导航雷达整机对外接口协议_2rd---改.docx 仅用于追溯历史定义；当其与本文不一致时，以当前代码实现为准。

1. 范围和接口

当前显控实际实现并使用以下两路 UDP 接口：

接口	方向	用途	当前代码入口
接口 1	显控 -> 雷达/伺服	下发雷达参数和转台命令	MarineRadarManager::sendControl()
接口 4	雷达/伺服 -> 显控	上报雷达状态和一条方位线的回波幅度	MarineRadarManager::parseEchoDatagram()

Word 中描述的雷达阵面内部接口、伺服与阵面之间的内部接口，不是当前 DispCtrl 进程收发的协议，本文不定义其报文。

2. 网络配置

配置文件使用以下字段：

```
[marine.network]
local_ip = "192.168.1.100"
echo_port = 9000
servo_ip = "192.168.1.30"
servo_port = 9000
auto_send_ms = 0
```

字段	含义	现行代码行为
local_ip	显控本机 IPv4 地址	在此地址绑定回波接收 Socket；空字符串时绑定 0.0.0.0。
echo_port	回波接收端口	显控绑定此 UDP 端口接收设备上报。
servo_ip	雷达/伺服目的 IPv4 地址	控制帧的目标地址。
servo_port	雷达/伺服目的端口	控制帧的目标端口。
auto_send_ms	自动下发周期，单位 ms	<= 0 时关闭周期发送；大于 0 时按该周期重复发送当前控制帧。

正常情况下，控制帧也通过已经绑定 `local_ip:echo_port` 的接收 Socket 发出，因此其源端口通常是 `echo_port`。这样可兼容“设备将回波回复到控制帧源端口”的实现。只有接收 Socket 未成功绑定时，才退回到未绑定发送 Socket，此时操作系统可能分配临时源端口。

当前接收端按本地绑定地址和端口接收 UDP 数据报，**不额外校验发送方 IP 或端口**。

3. 接口 1：显控 -> 雷达/伺服控制帧

3.1 基本约定

- 传输层：UDP。
- 帧长：固定 **16 字节**。
- 帧头：`0xA5`；帧尾：`0x5A`。
- `uint16_t Azimuth` 按小端发送，即低字节在前、高字节在后。
- 校验：`Byte[0]` 到 `Byte[13]` 的逐字节 XOR，结果写入 `Byte[14]`。
- 每一帧始终包含完整的雷达参数字段；`CMDNum` 决定本帧是否同时执行转台动作。

3.2 字节布局

字节偏移	字段	类型/字节序	取值和现行含义
0	HeadFlag	uint8	固定 <code>0xA5</code> 。
1	CMDNum	uint8	转台命令，见下表。
2-3	Azimuth	uint16，小端	位置模式方位，单位为角度乘 100。
4	RangeVal	uint8	量程编号 <code>0..14</code> 。
5	Gain	uint8	波束锐化：UI 当前使用 <code>0=关</code> ， <code>1=低</code> ， <code>2=中</code> ， <code>3=高</code> 。
6	GanRao	uint8	同频干扰抑制：UI 当前使用 <code>0=关</code> ， <code>1=低</code> ， <code>2=中</code> ， <code>3=高</code> 。
7	Level	uint8	截位/电平选择； <code>0=自动</code> ，其余值由手动滑块传递。
8	SeaVal	uint8	海杂波； <code>0=自动</code> ，其余值由手动滑块传递。
9	RainVal	uint8	雨雪杂波； <code>0=自动</code> ，其余值由手动滑块传递。
10	CFAR	uint8	保留。显控当前固定为 <code>0</code> 。
11	TXCtrl	uint8	发射控制： <code>0=关</code> ，非零=开；显控下发 <code>0</code> 或 <code>1</code> 。
12	Servo	uint8	转速档位 <code>0..8</code> 。
13	MTD	uint8	保留。显控当前固定为 <code>0</code> 。
14	Checksum	uint8	<code>Byte[0..13]</code> 的 XOR。
15	TailFlag	uint8	固定 <code>0x5A</code> 。

3.3 CMDNum 定义

值	名称	设备侧预期动作
0x00	ParamsOnly	仅更新雷达参数，不执行转台 UART 命令。
0x01	Position	更新雷达参数，并按 Azimuth 进入位置模式。
0x02	Speed	更新雷达参数，并按 Servo 档位进入速度模式。
0x03	Stop	更新雷达参数，并停止转台。
0x04	Start	更新雷达参数，并启动转台。

显控界面将 0x00、0x01、0x02 放在命令下拉框中；0x03 和 0x04 通过“停止”和“启动”按钮下发。无论命令为何值，显控都会构造并发送完整 16 字节帧。

3.4 方位和转速换算

控制帧方位不是 8192 方位编号。控制下发仍使用“角度乘 100”：

```
azimuthRaw = round(normalize(angleDeg, 0..360) * 100)
angleDeg    = azimuthRaw / 100.0
```

例如 180.00 deg：

```
azimuthRaw = 18000 = 0x4650
Byte[2]    = 0x50
Byte[3]    = 0x46
```

Servo 为速度档位，代码中定义：

```
rpm = min(Servo, 8) * 6.00
```

Servo	转速
0	0.00 rpm
1	6.00 rpm
2	12.00 rpm
3	18.00 rpm
4	24.00 rpm
5	30.00 rpm
6	36.00 rpm
7	42.00 rpm

Servo 转速

8 48.00 rpm

3.5 量程编号

当前代码只定义 0..14 共 15 档；超出该范围的回波帧会被丢弃，控制界面也不会产生超出范围的编号。

编号 最大显示量程

0 300 m

1 500 m

2 750 m

3 1 km

4 1.5 km

5 2 km

6 3 km

7 4 km

8 6 km

9 10 km

10 15 km

11 30 km

12 40 km

13 60 km

14 75 km

3.6 控制帧示例

仅更新雷达参数，量程 4 km、发射开、其余参数为 0：

```
A5 00 00 00 07 00 00 00 00 00 00 01 00 00 A3 5A
```

设置 3 档速度，同时下发同一组雷达参数：

```
A5 02 00 00 07 00 00 00 00 00 00 01 03 00 A2 5A
```

该帧中的 Servo=3 对应 18.00 rpm。控制 Socket 的 writeDatagram() 成功只表示显控已将 UDP 数据交给本机网络栈；当前协议实现没有控制确认/应答帧，不能据此确认设备已执行。

4. 接口 4：雷达/伺服 -> 显控回波帧

4.1 基本约定

- 传输层：UDP。
- 固定头长度：**22 字节**。
- 报文总长度：至少 **22 + fftWordCount * 4** 字节；额外尾随字节当前不会导致该报文被拒绝。
- 方位字段为 **8192 分辨率方位编号**，小端。
- **fftDataLen**、**packetNum**、**reserved** 均为大端。
- 每个距离单元为一个 **4 字节小端无符号幅值**。

4.2 字节布局

字节偏移	字段	字节序	现行解析规则
0	leadByte	-	必须为 0x00。
1	headFlag	-	必须为 0xA5。
2-3	aziLow、aziHigh	小端	方位编号 bin = Byte[2] + Byte[3] * 256。
4	style	-	仅接受 0x00、0x01、0x02。
5	hdrChecksum	-	Byte[0..4] 的 XOR。
6	hdrTail	-	必须为 0x5A。
7	rangeCode	-	当前量程编号，必须为 0..14。
8	txState	-	非零视为发射开。
9	gain	-	当前增益状态。
10	level	-	当前电平/截位状态。
11	seaVal	-	当前海杂波状态。
12	rainVal	-	当前雨杂波状态。
13	ganRao	-	当前同频干扰抑制状态。
14	freqStatus	-	当前频率状态。
15	statusChecksum	-	Byte[7..14] 的 XOR。
16-17	fftDataLenHigh、fftDataLenLow	大端	fftWordCount，单位为 4 字节距离单元。
18-19	packetNumHigh、packetNumLow	大端	包序号。当前显控记录并传递该值，不用其做丢包重排。
20-21	reservedHigh、reservedLow	大端	保留。

字节偏移	字段	字节序	现行解析规则
22 起	回波幅值	每单元 4 字节小端	共 <code>fftWordCount</code> 个距离单元。

4.3 方位换算和显示映射

设备上报的方位编号定义为：

```
bin      = Byte[2] + Byte[3] * 256
angleDeg = (bin mod 8192) * 360 / 8192
```

例：`Byte[2..3] = 00 08`，则 `bin=2048`，对应 `90.00 deg`。

PPI 渲染器内部为 4096 方位，因此显控进一步换算：

```
renderIndex = ((bin mod 8192) * 4096) / 8192
```

即 8192 协议方位的相邻两个编号会映射到同一个 4096 渲染方位。该映射只影响显控内部存储和绘制，不改变设备协议格式。

4.4 幅值和长度

```
echoBytes    = fftWordCount * 4
expectedSize = 22 + echoBytes
```

现行代码接受的 `echoBytes` 必须满足：

```
4 <= echoBytes <= 8192
echoBytes % 4 == 0
```

因此有效距离单元数为 **1 到 2048**。第 `i` 个单元的原始幅值按小端读取：

```
magnitude = b0 + (b1 << 8) + (b2 << 16) + (b3 << 24)
displayAmplitude = min(magnitude, 255)
```

显控渲染缓存目前是 8 位幅值，故大于 `255` 的原始值会饱和为 `255`；这不是网络报文截断，而是显示端的幅值压缩行为。

4.5 接收校验和拒收规则

显控会直接丢弃以下回波报文：

- 1. 总长度小于 22 字节。
- 2. `leadByte`、`headFlag`、`hdrTail` 或头部 XOR 校验不正确。
- 3. `style` 不属于 `0x00`、`0x01`、`0x02`。
- 4. `rangeCode` 不属于 `0..14`。
- 5. `fftDataLen` 推导出的回波长度不在 4 到 8192 字节范围内，或不是 4 的整数倍。
- 6. UDP 数据报实际长度小于 `22 + fftWordCount * 4`。

`statusChecksum` 不匹配时，当前代码记录告警，但仍继续解析状态和回波数据。方位编号超过 8191 时，当前代码采用 `mod 8192` 回绕后继续绘制。

5. 状态反馈与显示侧行为

每一条有效回波线都携带状态字段。显控将 `rangeCode`、`txState`、`gain`、`level`、`seaVal`、`rainVal`、`ganRao` 和 `freqStatus` 提取为运行状态；只有其中任一值变化时才向界面发布状态更新。

有效回波线随后被发送给 `EchoRenderer::updateEchoLine()`。以下项目属于显示端行为，而不是外部 UDP 字段：

配置	作用	默认值
<code>marine.display.echo_decay_ms</code>	回波余辉时长	<code>10000 ms</code>
<code>marine.display.colormap</code>	色图	<code>simrad</code>
<code>marine.display.sweep_history_rounds</code>	同一显示方位保留的扫描圈数	<code>1</code> ，范围 <code>1..8</code>
<code>marine.display.echo_debug</code>	接收和历史圈调试日志	<code>false</code>

量程切换会同步 PPI 几何范围和回波渲染范围。回波帧中的 `rangeCode` 同时用于确定该条回波数据对应的物理量程；因此设备在量程切换后应在上报帧中及时携带新的 `rangeCode`。

6. 与旧 Word 版本的主要差异

项目	当前代码实现	使用旧 Word 联调时需注意
<code>CMDNum</code>	已定义并生效， <code>0x00..0x04</code> 分别对应参数、位置、速度、停止、启动。	不能再将该字段视为保留。
控制方位	<code>角度 * 100</code> ，16 位小端。	这是控制下发格式，不是 8192 编号。
回波方位	8192 编号，小端；显控内部映射到 4096 渲染方位。	不能按“角度 * 100”解析回波 <code>Byte[2..3]</code> 。
量程表	仅 <code>0..14</code> ，300 m 到 75 km。	不能按旧 24 档表下发或上报。
回波固定头	实际结构体和断言为 22 字节。	不以历史文档中的头长度描述为准。
回波幅值	每距离单元 4 字节小端 <code>uint32</code> ；显示端饱和压缩为 8 位。	设备侧需按 4 字节单元计算 <code>fftDataLen</code> 。

项目	当前代码实现	使用旧 Word 联调时需注意
控制确认	当前显控没有协议级 ACK 解析。	“发送成功”仅代表本地 UDP 写入成功。

7. 联调检查清单

- 1. 确认显控绑定的 `local_ip` 是实际网卡地址，而非另一网段或不存在的地址。
- 2. 设备应向 `local_ip:echo_port` 发送回波；控制帧目标为 `servo_ip:servo_port`。
- 3. 设备回波 Byte[0..6] 必须为 `00 A5 <aziLow> <aziHigh> <style> <xor> 5A`，其中 XOR 覆盖 Byte[0..4]。
- 4. `fftDataLen` 必须使用大端，并表示 4 字节距离单元的数量，不是字节数量。
- 5. 回波载荷中的每个幅值必须使用小端 4 字节整数；总载荷至少 4 字节、最多 8192 字节。
- 6. 每个有效回波帧应携带正确的 `rangeCode`；设备量程切换后不可继续长期上报旧量程编号。
- 7. 控制方向的 `Azimuth` 与回波方向的方位字段编码不同：前者为角度乘 100，后者为 8192 方位编号。

8. 代码定位

- 报文结构、常量、校验和单位：`Basic/MarineProtocol.h`
- UDP 绑定、控制发送、回波校验与解析：`Controller/MarineRadarManager.cpp`
- 网络和初始控制参数读取：`Controller/controller.cpp`、`config.toml`
- 界面控制命令与量程/速度取值：`mainPanel/mainoverlaylayout.cpp`
- 回波交给 PPI 渲染器的连接：`PolarDisp/ppisscene.cpp`